

Labwork 8.2 - Puzzles

General Information and Submission Rules:

- In this labwork, you need to design a machine to solve some secret mathematical puzzles.
- By the end of the labwork, you must submit it to Yulearn as **studentID_fullname_labwork8.c**.
- Only use concepts covered in the labs so far.

Question 1 - Display Message (10p)

- Create a new function (**greetings**) that displays a welcome message and a list of all available operations to the user.
- This function will expect a decision from a user so that it will execute related functions based on this choice until the user exits.

```
Welcome to the Number Games Machine! By using this machine you can do:
1. Double-way Palindrome
2. Signal Puzzle
3. Removing Digits
4. Exit
Make your choice:
```

Question 2 - Largest Palindrome (30p)

- The decimal number, 585 = 10010 01001 is palindromic in both cases.
- Find the sum of all numbers, less than one million, which are palindromic in base 10 and base 2.
- You can write a helper function to convert a decimal number to binary (**decimal_to_binary**) or otherwise.

```
The total sum is 872187.
```

Question 3 - Signal Puzzle (30p)

- Imagine a satellite sending “data packets” to Earth. Each packet is a string of numbers. To ensure the data isn’t corrupted, the satellite follows these rules, by using these rules write a function (**signal_puzzle**):
 - Every packet has a “Checksum” value. If the sum of all numbers in a packet is **even**, the packet is “Stable / 1”. If it is **odd**, it is “Unstable / 0”.
 - A “Burst” occurs if a number in the packet is a prime number. Prime numbers cause the signal to jump.
- You are given a sequence of packets (arrays of integers). You need to find out how many “Perfect Signals” occurred.
- A “Perfect Signal” is a packet that is stable (even sum) and contains exactly two prime numbers.
- You can write two separate helper function: one for determine prime number (**is_prime**) and one for determine the stable signal (**is_stable**)

```
int satellite_data[3][4] = { {2, 3, 4, 6}, {10, 7, 5, 2}, {4, 11, 8, 13} };
```

```
Not Perfect  
Not Perfect  
Perfect
```

Question 4 - Removing Digits (30p)

- Write a function named (**removing_digits**) that takes one digit in each call.
- This game starts with a positive integer.
- Two players take turns to remove a single digit from that integer.
- The winner is the person who removes the last nonzero digit.
- For example, removing a digit from 105 results in either 5, 10 or 15.

```
int number = 872187;
```

```
>> Enter the digit: 7  
82187  
>> Enter the digit: 1  
8287  
>> Enter the digit: 8  
287  
>> Enter the digit: 8
```

27

>> Enter the digit: 7

2

>> Enter the digit: 2

0

Winner: First Player